

School Management App

Form Request Réteg – Fejlesztői dokumentáció

Laravel 12 + Clean Code elvek | 2026

1. Mi a Form Request és miért kell?

A Form Request egy speciális Laravel osztály, amelynek két felelőssége van: jogosultság ellenőrzés (authorize), és validáció (rules). Mindkettő automatikusan lefut mielőtt a Controller metódusa elindulna.

1.1 A probléma – validáció a Controllerben

```
// ROSSZ - Controller túl sokat csinál
public function store(Request $request)
{
    // Jogosultság ellenőrzés
    if (!auth()->user()->isAdmin()) {
        return response()->json(['error' => 'Forbidden'], 403);
    }

    // Validáció
    $request->validate([
        'name'      => 'required|string|max:255',
        'email'     => 'required|email|unique:students',
        'class_id' => 'required|exists:classes,id',
    ]);

    // ... és még a tényleges logika sem kezdődött el
}
```

1.2 A megoldás – Form Request

```
// JÓ - Controller csak delegál
public function store(StoreStudentRequest $request)
{
    // A validáció és jogosultság már le van ellenőrizve
    // mire idáig ér a kód - automatikusan!
    $student = $this->studentService->createStudent($request-
    >validated());
    return new StudentResource($student);
}
```

```
}
```

1.3 Hogyan működik automatikusan?

A Laravel felismeri a típusjelzésből (type-hint) hogy a metódus `StoreStudentRequest`-et vár. Mielőtt a Controller metódus lefut, három lépés történik:

1. Lefuttatja az `authorize()` metódust → `false` esetén 403-as hiba (Forbidden)
2. Lefuttatja a `rules()` metódust → hibás adat esetén 422-es hiba (Unprocessable)
3. Ha minden rendben → átadja a `validated()` adatokat a Controllernek

2. A Form Request osztály metódusai

Metódus	Visszatérés	Felelőssége
<code>authorize()</code>	bool	Végrehajthatja-e a felhasználó ezt a műveletet
<code>rules()</code>	array	Validációs szabályok minden mezőhöz
<code>messages()</code>	array	Egyedi (magyar) hibaüzenetek
<code>withValidator()</code>	void	Extra validáció, amit <code>rules()</code> nem tud kezelni

3. Store vs Update – a 'sometimes' szabály

Minden modulhoz két Form Request készül: Store (létrehozás) és Update (módosítás). A kettő között a legfontosabb különbség a 'sometimes' szabály használata.

3.1 StoreStudentRequest – minden mező kötelező

```
public function rules(): array
{
    return [
        'name'          => 'required|string|max:255',
        'email'         => 'required|email|unique:students,email',
        'phone'         => 'nullable|string|max:20',
        'address'       => 'nullable|string|max:500',
        'date_of_birth' => 'nullable|date|date_format:Y-m-d|before:today',
    ];
}
```

```

        'gender'          => 'nullable|in:male,female,other',
        'class_id'        => 'required|exists:classes,id',
    ];
}

```

Mit jelentenek ezek a szabályok?

required = kötelező mező. nullable = lehet null/hiányzó. email = érvényes email formátum kell. unique:students,email = az email egyedi kell legyen a students táblában. date_format:Y-m-d = a dátum YYYY-MM-DD formátumú kell legyen. before:today = csak múltbeli dátum (pl. születési dátum nem lehet jövőbeli). in:male,female,other = csak ezek közül választhat. exists:classes,id = az id-nek léteznie kell a classes táblában.

3.2 UpdateStudentRequest – sometimes és unique kivétel

```

public function rules(): array
{
    $studentId = $this->route('student');

    return [
        'name'          => 'sometimes|required|string|max:255',
        'email'          =>
"sometimes|required|email|unique:students,email,{ $studentId }",
        'phone'          => 'nullable|string|max:20',
        'class_id'        => 'sometimes|required|exists:classes,id',
    ];
}

```

Miért 'sometimes|required' és nem csak 'required'?

A sometimes azt jelenti: a mező csak akkor validálódik, ha SZEREPEL aérésben. Ha a frontend csak az email-t küldi frissítésre, a name mező validációja nem fut le – mert nem küldték. De HA küldik, akkor required (nem lehet üres string). Ez teszi lehetővé a részleges frissítést (PATCH jellegű működés).

Miért kell a unique szabálynál a { \$studentId }?

A unique:students,email,{ \$studentId } azt jelenti: az email egyedi kell legyen, KIVÉVE annál a rekordnál amelynek id-ja \$studentId. Nélküle a saját email címére is 'már foglalt' hibát kapnánk frissítéskor – hiszen az adatbázisban már szerepel a saját email cím a saját rekordjában. A \$this->route('student') az URL-ből veszi ki az id-t, pl. /api/students/5 → 5.

4. Tömb (array) validáció – Attendance

Az osztály jelenlétének rögzítésekör egyszerre több diák adatát küldjük – ez egy tömb aérésben. A Laravel speciális szintaxissal validálja a tömb minden elemét.

4.1 StoreAttendanceRequest – diák jelenlét

```
public function rules(): array
{
    return [
        'class_id' => 'required|exists:classes,id',
        'date'      => 'required|date|date_format:Y-m-d',

        'attendances' => 'required|array|min:1',
        'attendances.*.student_id' => 'required|exists:students,id',
        'attendances.*.status' =>
        'required|in:present,absent,late,excused',
        'attendances.*.note' => 'nullable|string|max:255',
    ];
}
```

Hogyan kell olvasni a tömb validációs szintaxist?

'attendances' => 'array|min:1' → maga a tömb kötelező, legalább 1 elemmel.

'attendances.*.student_id' → a csillag (*) azt jelenti: a tömb MINDEN elemének student_id mezője. Tehát ha 30 diákot küldünk, mind a 30 elem student_id és status mezője validálódik ugyanezekkel a szabályokkal.

A frontend így küldi az adatot:

```
{
  "class_id": 1,
  "date": "2026-06-05",
  "attendances": [
    {"student_id": 1, "status": "present", "note": ""},
    {"student_id": 2, "status": "absent", "note": "beteg"}
  ]
}
```

4.2 StoreTeacherAttendanceRequest – tanár jelenlét

Az attendances tábla teacher_id mezője is nullable – tehát tanárok jelenléte is rögzíthető. Ehhez egy külön Request készült, mert a mezőnév eltér (teacher_id helyett student_id), és a két fogalom logikailag is külön végpont.

```
public function rules(): array
{
    return [
        'class_id' => 'required|exists:classes,id',
```

```

        'date'          => 'required|date|date_format:Y-m-d',

        'attendances'    => 'required|array|min:1',
        'attendances.*.teacher_id' => 'required|exists:teachers,id',
        'attendances.*.status' =>
        'required|in:present,absent,late,excused',
        'attendances.*.note' => 'nullable|string|max:255',
    ];
}

```

5. Mezők közötti összefüggések – lte és after

Néhány mező értéke egy másik mező értékétől függ. A Laravel beépített szabályai (lte, after) ezt egyszerű esetekben kezelik – de Update-nél finomítani kell rajtuk.

5.1 StoreExamRequest – passing_marks lte total_marks

```

public function rules(): array
{
    return [
        'total_marks' => 'required|integer|min:1',

        // lte = less than or equal to (kisebb vagy egyenlő)
        // A 'total_marks' a KÉRÉSBEN lévő mező nevére hivatkozik
        'passing_marks' => 'required|integer|min:1|lte:total_marks',

        'start_time' => 'nullable|date_format:H:i',
        'end_time' => 'nullable|date_format:H:i|after:start_time',
    ];
}

```

Mit csinál az lte: és after:?

lte:total_marks → a passing_marks értéke kisebb vagy egyenlő kell legyen mint a total_marks értéke UGYANEKBEN a kérésben. after:start_time → az end_time később kell legyen mint a start_time. Mindkét szabály a KÉRÉSBEN szereplő másik mező értékét hasonlítja össze – ez Store esetén tökéletes, mert mindkét mező mindig jelen van.

5.2 UpdateExamRequest – amikor az lte: nem elég

Update esetén előfordulhat hogy csak a passing_marks érkezik a kérésben, a total_marks nem (mert azt nem módosítja a felhasználó). Ekkor az lte:total_marks nem tudna mit kezdeni – a kérésben nincs total_marks mező.

```

public function rules(): array
{
    return [
        'total_marks' => 'sometimes|required|integer|min:1',
        'passing_marks' => 'sometimes|required|integer|min:1',
        // figyeld meg: itt NINCS lte:total_marks!
    ];
}

public function withValidator($validator): void
{
    $validator->after(function ($validator) {
        if ($this->filled('passing_marks')) {
            $exam = $this->route('exam');

            // Ha jön total_marks a kérdésben, azt használjuk,
            // ha nem, az ADATBÁZISBAN lévő értéket vesszük
            $totalMarks = $this->input('total_marks') ?? $exam-
>total_marks;

            if ($this->input('passing_marks') > $totalMarks) {
                $validator->errors()->add(
                    'passing_marks',
                    'Az átmenő pontszám nem lehet nagyobb mint ' .
                    $totalMarks
                );
            }
        }
    });
}

```

Miért kell ez a bonyolultabb megoldás?

Az lte:total_marks csak a KÉRÉSBEN lévő mezőt tudja nézni – ha az nincs jelen, a szabály nem tud összehasonlítani. A withValidator()-ban viszont elérjük az ADATBÁZIST is (\$exam->total_marks), így mindkét esetet helyesen kezeljük: ha jön total_marks a kérdésben, azt; ha nem, az adatbázisban lévő (jelenlegi) értéket hasonlítjuk a passing_marks-hoz.

6. withValidator() – "vagy-vagy" logika

Az IssueLibraryBookRequest egy klasszikus XOR (kizáró vagy) esetet kezel: egy könyvet VAGY egy diák, VAGY egy tanár kölcsönöz – de nem mindkettő egyszerre, és nem is senki.

6.1 A probléma – rules() önmagában nem elég

A rules() tömb minden mezőt egymástól függetlenül validál. Nem tudja kifejezni azt, hogy 'A vagy B kötelező, de nem mindkettő'. Ehhez a withValidator() metódusra van szükség, amely a teljes validált adathalmazhoz hozzáfér.

6.2 IssueLibraryBookRequest – teljes megoldás

```
public function rules(): array
{
    return [
        'student_id' => 'nullable|exists:students,id',
        'teacher_id' => 'nullable|exists:teachers,id',
        'due_date'    => 'required|date|date_format:Y-m-d|after:today',
    ];
}

public function withValidator($validator): void
{
    $validator->after(function ($validator) {
        $hasStudent = $this->filled('student_id');
        $hasTeacher = $this->filled('teacher_id');

        if (!$hasStudent && !$hasTeacher) {
            $validator->errors()->add('student_id', 'Legalább egy diákot vagy tanárt meg kell adni.');
```

vagy tanárt meg kell adni.');

```
            $validator->errors()->add('teacher_id', 'Legalább egy diákot vagy tanárt meg kell adni.');
```

}

```
            if ($hasStudent && $hasTeacher) {
                $validator->errors()->add('student_id', 'Csak diákot vagy tanárt lehet megadni, nem mindkettőt.');
```

\$validator->errors()->add('teacher_id', 'Csak diákot vagy tanárt lehet megadni, nem mindkettőt.');

```
            }
        });
    });
}
```

Mit csinál a filled() és a validator->after()?

`$this->filled('student_id')` → true ha a mező létezik a kérésben ÉS nem üres (különbözik az `has()`-tól, ami akkor is true lenne ha üres stringet küldenék). A `validator->after()` egy callback-et regisztrál, amely az ALAP szabályok lefutása UTÁN fut le – itt már bármilyen egyedi logikát írhatunk, és az `errors()->add()` segítségével saját hibaüzeneteket adhatunk a validátorhoz.

Miért mindkét mezőhöz adjuk hozzá a hibát?

Ha csak a `student_id`-hoz adnánk hibaüzenetet, a frontend csak azt az egy mezőt tudná pirossal kiemelni. Mindkét mezőhöz hozzáadva a frontend mindkét inputot megjelölheti hibásként – jobb felhasználói élmény.

7. Amit a Form Request szándékosan NEM validál

Nem minden adatbázis mező jelenik meg a Form Request-ben. Van mezők, amelyeket a Service réteg számít ki – ezeket NEM a felhasználó küldi, tehát validálni sem kell őket itt.

7.1 StoreExamResultRequest – grade és status hiánya

```
public function rules(): array
{
    return [
        'results' => 'required|array|min:1',
        'results.*.student_id' => 'required|exists:students,id',
        'results.*.marks_obtained' => 'required|integer|min:0',
        'results.*.remarks' => 'nullable|string|max:500',

        // grade és status NINCS itt!
    ];
}
```

Miért hiányzik szándékosan a grade és status?

Ezeket az ExamService számítja ki automatikusan a marks_obtained és a vizsga total_marks / passing_marks alapján: `$grade = calculateGrade(...); $status = $marks >= $passing ? 'pass' : 'fail';` Ha a Request validálná és elfogadná ezeket a mezőket, a frontend (vagy egy rosszindulatú kérés) saját maga állíthatná be az osztályzatot – megkerülve az üzleti logikát. A frontend csak a marks_obtained-et küldi, a grade és status mindig a backend Service-ben generálódik.

8. Az összes Form Request áttekintése

Form Request	Modul	Különleges jellemző
StoreStudentRequest	Student	exists:classes,id (osztály léte)
UpdateStudentRequest	Student	sometimes + unique kivétellel
StoreTeacherRequest	Teacher	alap CRUD validáció
UpdateTeacherRequest	Teacher	sometimes + unique kivétellel
StoreClassRequest	SchoolClass	nullable osztályfőnök
UpdateClassRequest	SchoolClass	sometimes minden mezőn
StoreSubjectRequest	Subject	unique kód (code)
UpdateSubjectRequest	Subject	sometimes + unique kivétellel
StoreAttendanceRequest	Attendance	tömb validáció (diákok)
StoreTeacherAttendanceRequest	Attendance	tömb validáció (tanárok)
StoreExamRequest	Exam	lte:total_marks, after:start_time

UpdateExamRequest	Exam	withValidator + adatbázis lekérdezés
StoreExamResultRequest	ExamResult	grade/status szándékosan hiányzik
StoreFeeRequest	Fee	decimal:0,2 pénzösszeg validáció
UpdateFeeRequest	Fee	sometimes minden mezőn
StoreLibraryBookRequest	Library	lte:total_copies
IssueLibraryBookRequest	Library	withValidator XOR logika
StoreNotificationRequest	Notification	enum mezők (type, target, channel)

9. Összefoglaló – Validációs szabályok szótára

Szabály	Jelentés	Példa a projektből
required	Kötelező mező	name, email
sometimes	Csak ha jelen van a kérésben	Update Requestek
nullable	Lehet null / hiányzó	phone, address
string / integer / numeric	Adattípus ellenőrzés	name (string), total_marks (integer)
max: / min:	Karakter- vagy szám-korlát	max:255, min:1
email	Érvényes email formátum	email mezők
unique:tábla,oszlop,kivétel	Egyedi érték az adatbázisban	email, code, isbn
exists:tábla,id	A hivatkozott id létezik	class_id, teacher_id
in:a,b,c	Csak felsorolt értékek (enum)	gender, status, type
date / date_format:Y-m-d	Dátum formátum ellenőrzés	date_of_birth, due_date
before:today	Csak múltbeli dátum	date_of_birth
after:today	Csak jövőbeli dátum	due_date (library)
after:mező	Másik mező értéke után	end_time after:start_time
lte:mező	Kisebb vagy egyenlő mint másik mező	passing_marks lte:total_marks
array / min:1	Tömb, legalább 1 elemmel	attendances, results
mező.*.al_mező	Tömb minden elemének validálása	attendances.*.student_id
decimal:0,2	Tizedesjegyek száma	amount (Fee)

Form Request mappa struktúra

```
app/  
└─ Http/  
    └─ Requests/  
        ├── StoreStudentRequest.php  
        ├── UpdateStudentRequest.php  
        ├── StoreTeacherRequest.php  
        ├── UpdateTeacherRequest.php  
        ├── StoreClassRequest.php  
        ├── UpdateClassRequest.php  
        ├── StoreSubjectRequest.php  
        ├── UpdateSubjectRequest.php  
        ├── StoreAttendanceRequest.php  
        ├── StoreTeacherAttendanceRequest.php  
        ├── StoreExamRequest.php  
        ├── UpdateExamRequest.php  
        ├── StoreExamResultRequest.php  
        ├── StoreFeeRequest.php  
        ├── UpdateFeeRequest.php  
        ├── StoreLibraryBookRequest.php  
        ├── IssueLibraryBookRequest.php  
        └─ StoreNotificationRequest.php
```

– School Management App fejlesztési dokumentáció –